

## **Tracking U.S. Housing Market Trends with InfluxDB**

Abigail Eng, Colin Hui, Emily Quach, Rau-Shawn Tribuce, Tam Vu

Northeastern University

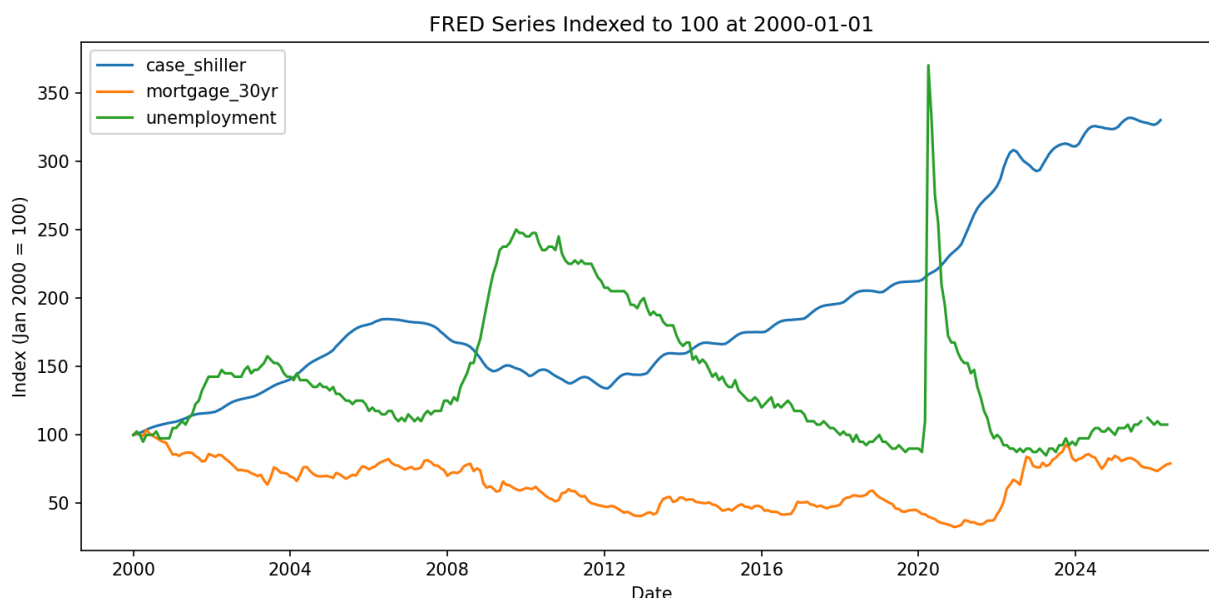
DS4300 - Homework 5

Professor Rachlin

June 15, 2026

## Abstract

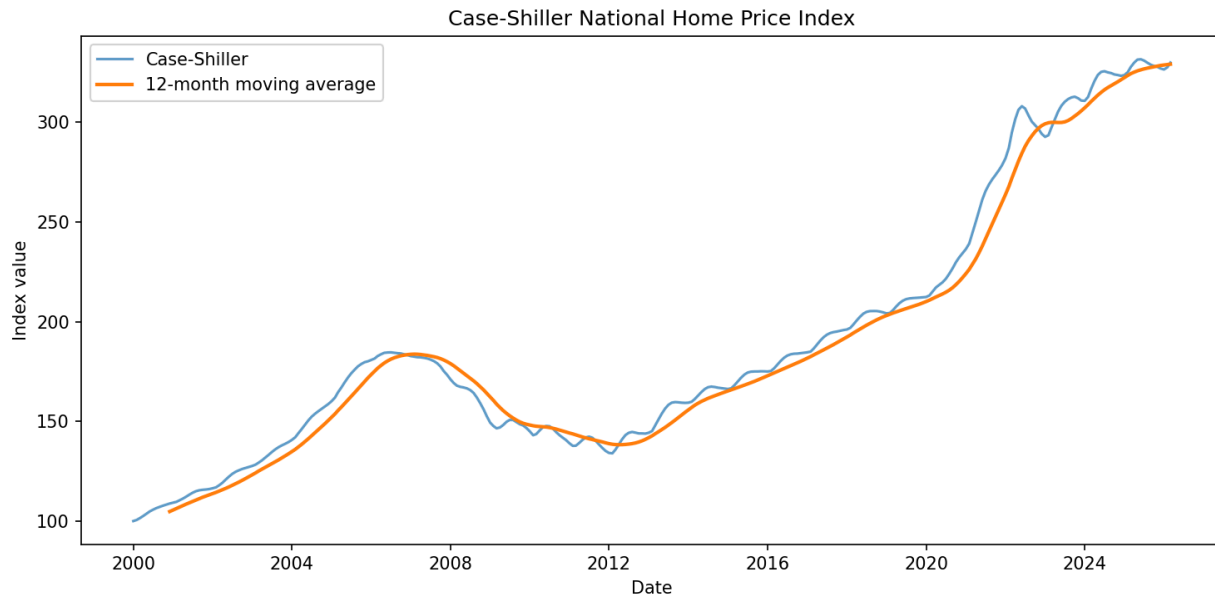
The conflict between housing affordability and price volatility became the central focus of our analysis. We chose InfluxDB to store query results and analyze historical median housing prices from the Federal Reserve Economic Data (FRED) because it's well-suited for time-series NoSQL data. Using Python as our primary programming language, we explored timestamped housing price data stored in InfluxDB to create queries that revealed trends and patterns in the housing market. Our analysis revealed that U.S. home prices almost tripled between 2000 and 2025, but in distinct phases instead of a steady climb. As we can see in the visualizations, there is a sharp decline in home prices after the 2008 housing crisis, followed by a slow recovery, and then a steep increase post-COVID. We also saw that home prices moved inversely to mortgage rates for most of the period. However, this relationship broke down in 2022 when prices continued to increase even as rates spiked during the Fed's rate-hiking cycle. This investigation demonstrated that InfluxDB is an excellent tool for economic time-series analysis. Not only does it offer a wide range of features, but it also provides an intuitive data modeling approach that traditional relational databases, such as SQL, would find challenging.



## Introduction

The most effective way for the average American to build wealth is through real estate ownership. It serves as a crucial economic indicator of financial well-being, influencing the average household wealth and consumer confidence, which in turn impacts Federal Reserve policy decisions. Given our shared background in Economics, Finance, and Mathematics, this was a significant area of interest for us, as it has far-reaching effects on the overall economy. We aimed to identify insights from the data that could explain events like the 2008 housing crisis. We chose InfluxDB as our NoSQL tool because the foundation of most of our data is time series. InfluxDB enables us to measure price fluctuations month-to-month, year-over-year, and across different cities and market conditions. Traditional databases like MySQL or PostgreSQL can store this data, but weren't designed to use time as a parameter. This necessitates workarounds for range queries and downsampling. InfluxDB is specifically designed to handle this type of

workflow, which is why our team opted for it in our analysis.



InfluxDB was founded in 2012 by its parent company, InfluxDATA, to solve a problem that traditional databases never really handled well, which is storing massive amounts of timestamped data with efficiency and speed in mind. The whole database is organized around four building blocks: measurements, tags, fields, and timestamps. Instead of scanning each row after a query is performed in MySQL, InfluxDB can jump to the right position in a time-ordered column, which cuts down the processing time drastically. As for scalability, it can handle up to 10 million data points per second, which means it works well for larger companies such as Capital One and Tesla. Influx is also completely free and open-sourced under MIT and Apache 2 licenses, with paid expertise tiers for companies that need clustering and more processing. The community is very active, with over 5,000 companies utilizing it in production, and the GitHub repo has nearly 30,000 stars. For a database that's only been around for the past decade, it says a lot. This report will provide a comprehensive overview of our entire process, including the reasons behind our choice of InfluxDB, the structure and investigation of our chosen price dataset, the queries and visualizations built in Python, and a comprehensive review of where InfluxDB outperforms NoSQL databases and their associated challenges.

## Use-Cases:

### Case 1: Capital One

A large bank like Capital One, which serves more than 100 million customers, generates an enormous amount of data at any given moment. (Capital One, n.d.) Since data systems are a critical part of banking operations, Capital One uses InfluxDB to monitor its internal infrastructure, servers, and business metrics to track errors and analyze patterns for fraud detection and customer behavior. (InfluxData, n.d.a)

Some examples of the questions their systems need to answer in real time:

- How many milliseconds does it take for the login page to respond?
- How many credit card transactions are being processed per second?

- How many "500 Internal Server Error" responses are occurring per minute?
- Are there any unusual spikes in failed login attempts that might signal a fraud attack?
- How much data is flowing in and out of their network per second?
- How many transactions got flagged by the fraud detection system per hour?

Because all of these are time-series data, where each entry is indexed by a timestamp, querying and analyzing them is most efficient when the data is stored in a time-ordered, columnar structure. Since InfluxDB locates values by jumping directly to the corresponding position in the value column rather than scanning every row, searching takes a fraction of the time compared to a relational database or document-based format like JSON. For a financial institution like Capital One, speed is everything: every minute of downtime or app failure directly impacts customers' daily transactions, which in turn affects revenue and reputation. Being able to identify and diagnose a system issue within seconds rather than minutes is exactly what InfluxDB is built for.

## Case 2: Tesla

As a manufacturing enterprise, Tesla's priorities are operational efficiency, machinery productivity, and minimizing downtime. Like any large-scale manufacturer, keeping machines running at peak performance is essential because if a production line stops unexpectedly, it directly cuts into output and revenue.

This is where time-series data becomes critical. By tagging each data point with dimensions like sensor type, machine ID, temperature, or power consumption, Tesla engineers can track every piece of equipment continuously over time. Instead of waiting for a machine to break down, they can spot warning patterns early. For example, a motor whose temperature has been steadily climbing over the past 48 hours, and schedule maintenance before a failure actually occurs. InfluxDB makes this kind of analysis fast and precise, allowing engineers to pinpoint exactly when and where a problem is developing. The result is less unplanned downtime, lower maintenance costs, and a production line that keeps running. (InfluxData, n.d.b)

## Tutorial: Using InfluxDB 3 Core for Time-Series Analysis

**Setup and Installation:** InfluxDB 3 Core runs on Windows, macOS, and Linux and is available for free under the MIT and Apache 2 open source licenses. On Windows systems, installation requires downloading the binary from [influxdata.com](https://influxdata.com) and extracting it to a local directory. The server is started with a single PowerShell command:

```
.\influxdb3.exe serve --node-id my_host --object-store file --data-dir C:\influxdb3\data
```

```
PS C:\influxdb3> .\influxdb3.exe serve --node-id my_host --object-store file --data-dir C:\influxdb3\data
```

This command starts a local InfluxDB instance on port 8181, storing data as Apache Parquet files on the local filesystem. The server is ready to accept connections within seconds. Once running, an admin authentication token is created, and a database is initialized:

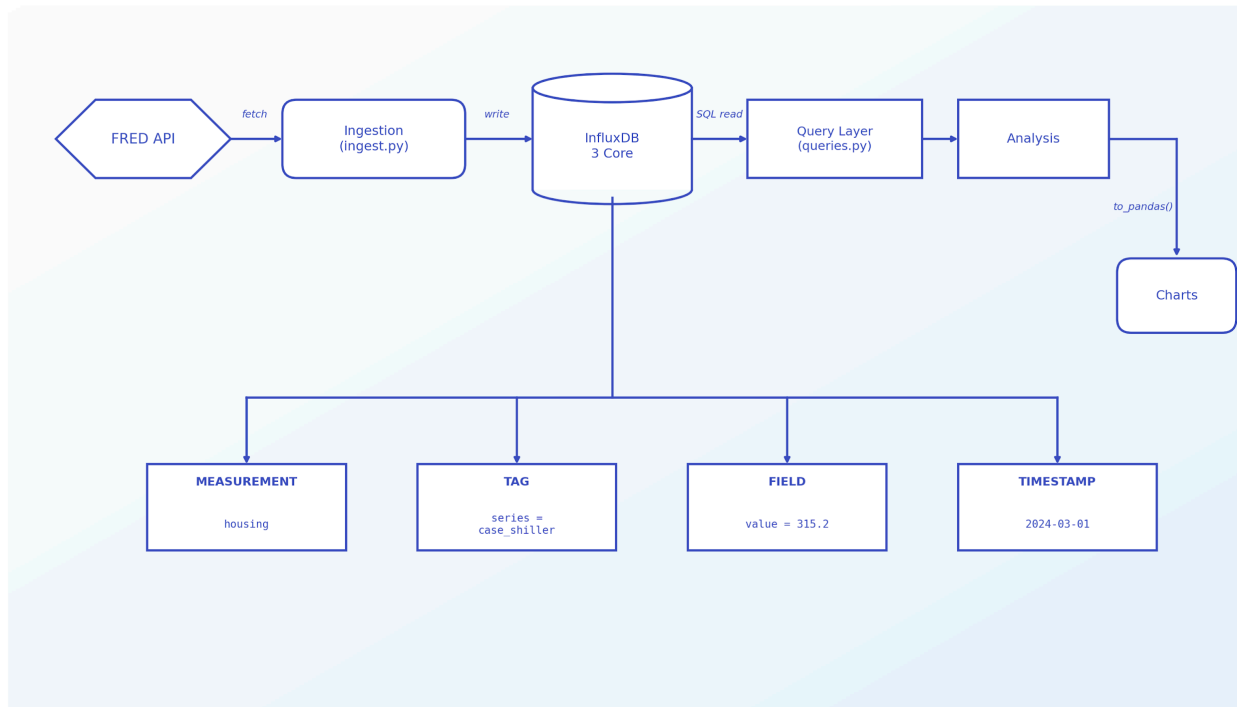
```
.\influxdb3.exe create token --admin
```

```
PS C:\influxdb3> .\influxdb3.exe create token --admin
```

```
.\influxdb3.exe create database housing --token <your_token>
```

```
PS C:\influxdb3> .\influxdb3.exe create database housing --token YOUR_TOKEN_HERE
```

The token is used to authenticate all subsequent reads and writes. The database named housing will store all three of our FRED time series.



**Data Model:** Before writing data, it is important to understand InfluxDB's core data model, which differs significantly from both relational databases and other NoSQL systems. InfluxDB organizes data into four building blocks:

- **Measurement:** analogous to a table, the demo version is called housing.
- **Tags:** indexed key-value string metadata used for filtering and grouping. We use a single tag called series to identify which dataset each point belongs to (ex. *case\_shiller*, *mortgage\_30yr*, *unemployment*).
- **Fields:** the actual numeric or string values being measured. We store a single field called value containing the numeric reading for each data point.
- **Timestamp:** Every point must have a timestamp. InfluxDB stores timestamps at nanosecond precision.

**Ingesting Data:** For this tutorial, we pull three macroeconomic time series from the Federal Reserve Economic Data (FRED) API using the fredapi Python library:

- **Case-Shiller National Home Price Index (CSUSHPINSA):** a monthly measure of U.S. residential home prices indexed to January 2000
- **30-Year Fixed Mortgage Rate (MORTGAGE30US):** the weekly national average 30-year fixed mortgage rate, resampled to monthly
- **Unemployment Rate (UNRATE):** the monthly U.S. national unemployment rate

The tutorial ingest script fetches each series starting from January 2000, resamples to monthly frequency, and writes each data point to InfluxDB as a Point object:

```
def fetch_series(fred, series_id): 1 usage
    raw = fred.get_series(series_id, observation_start=START_DATE)
    monthly = raw.resample("MS").mean().dropna()
    monthly.name = series_id
    return monthly

def write_to_influx(client, series_tag, monthly): 1 usage
    points = []
    for ts, val in monthly.items():
        point = (
            Point("housing")
            .tag(key="series", series_tag)
            .field("value", float(val))
            .time(ts)
        )
        points.append(point)
    client.write(record=points)
    return len(points)
```

Each data point is constructed with one tag (*series*) identifying which dataset it belongs to, one field (*value*) storing the numeric reading, and a timestamp. Running the ingest script produces the following output:

**Query Language:** InfluxDB has two supported query interfaces: InfluxQL, its original native query language, and SQL, which became the primary interface in InfluxDB 3. This shift towards standard SQL lowers the learning curve and improves compatibility with existing data tooling. InfluxQL is still supported for backwards compatibility but SQL is the recommended interface going forward.

**Example 1:** Basic SELECT

```
def example_select(client): 1 usage
    sql = """
        SELECT time, value
        FROM housing
        WHERE series = 'case_shiller'
        ORDER BY time DESC
        LIMIT 5
    """
    return run(client, sql)
```

```

Example 1: basic select (latest 5 Case-Shiller)
      time      value
0 2026-03-01  329.938
1 2026-02-01  327.592
2 2026-01-01  326.481
3 2025-12-01  326.968
4 2025-11-01  327.817

```

This query retrieves the 5 most recent Case-Shiller home price index values ordered by time descending. This shows InfluxDB's support for SQL syntax including ORDER BY and LIMIT for simple time-series lookups.

### Example 2: COUNT with GROUP BY

```

def example_count_by_series(client): 1 usage
    sql = """
        SELECT series, COUNT(*) AS n
        FROM housing
        GROUP BY series
    """
    return run(client, sql)

```

```

Example 2: count by series
      series      n
0 mortgage_30yr  318
1 case_shiller  315
2 unemployment  316

```

This aggregation query counts how many data points were loaded for each series, grouped by the series tag. It verifies that all three FRED datasets were ingested correctly and shows how InfluxDB's tag system enables filtering and grouping by metadata.

### Example 3: Time-bucketed sampling with date\_bin

```

def example_yearly_average(client): 1 usage
    sql = """
        SELECT date_bin(INTERVAL '1 year', time) AS year,
               AVG(value) AS avg_value
        FROM housing
        WHERE series = 'mortgage_30yr'
        GROUP BY year
        ORDER BY year
    """
    return run(client, sql)

```

Example 3: yearly average mortgage rate

|    | year       | avg_value |
|----|------------|-----------|
| 0  | 2000-01-01 | 8.061625  |
| 1  | 2001-01-01 | 6.971042  |
| 2  | 2002-01-01 | 6.536708  |
| 3  | 2003-01-01 | 5.819542  |
| 4  | 2004-01-01 | 5.838500  |
| 5  | 2005-01-01 | 5.864875  |
| 6  | 2006-01-01 | 6.410833  |
| 7  | 2007-01-01 | 6.340458  |
| 8  | 2008-01-01 | 6.036125  |
| 9  | 2009-01-01 | 5.041375  |
| 10 | 2010-01-01 | 4.690583  |
| 11 | 2011-01-01 | 4.455833  |
| 12 | 2012-01-01 | 3.655917  |
| 13 | 2013-01-01 | 3.981917  |
| 14 | 2014-01-01 | 4.172833  |
| 15 | 2015-01-01 | 3.849000  |
| 16 | 2016-01-01 | 3.647708  |
| 17 | 2017-01-01 | 3.991000  |
| 18 | 2018-01-01 | 4.539042  |
| 19 | 2019-01-01 | 3.933750  |
| 20 | 2020-01-01 | 3.113667  |
| 21 | 2021-01-01 | 2.955708  |
| 22 | 2022-01-01 | 5.327500  |
| 23 | 2023-01-01 | 6.795375  |
| 24 | 2024-01-01 | 6.723667  |
| 25 | 2025-01-01 | 6.595875  |
| 26 | 2026-01-01 | 6.267000  |

This query uses InfluxDB's `date_bin` function to bucket mortgage rate data into yearly intervals and compute the average for each year. This shows how InfluxDB allows users to downsample high-frequency time-series data to the desired resolution in a single query.

#### Example 4: Self-join across two series

```
def example_join_series(client): 1 usage
    sql = """
        SELECT cs.time AS time,
               cs.value AS case_shiller,
               mr.value AS mortgage_30yr
        FROM housing AS cs
        JOIN housing AS mr
        ON cs.time = mr.time
        WHERE cs.series = 'case_shiller'
        AND mr.series = 'mortgage_30yr'
        ORDER BY time
        LIMIT 10
    """
    return run(client, sql)
```

Example 4: join Case-Shiller + mortgage rate

|   | time       | case_shiller | mortgage_30yr |
|---|------------|--------------|---------------|
| 0 | 2000-01-01 | 100.000      | 8.2100        |
| 1 | 2000-02-01 | 100.571      | 8.3250        |
| 2 | 2000-03-01 | 101.466      | 8.2400        |
| 3 | 2000-04-01 | 102.540      | 8.1525        |
| 4 | 2000-05-01 | 103.701      | 8.5150        |
| 5 | 2000-06-01 | 104.856      | 8.2880        |
| 6 | 2000-07-01 | 105.723      | 8.1475        |
| 7 | 2000-08-01 | 106.523      | 8.0275        |
| 8 | 2000-09-01 | 107.137      | 7.9120        |
| 9 | 2000-10-01 | 107.729      | 7.7950        |

This query joins the housing table against itself to place Case-Shiller and mortgage rate values side by side on matching timestamps. The result enables correlation analysis between home prices and interest rates across the same time periods.

#### Example 5: Window function (12 month moving average)

```
def example_moving_average(client): 1 usage
    sql = """
        SELECT time,
               value,
               AVG(value) OVER (
                   ORDER BY time
                   ROWS BETWEEN 11 PRECEDING AND CURRENT ROW
               ) AS moving_avg_12mo
        FROM housing
        WHERE series = 'case_shiller'
        ORDER BY time
        LIMIT 20
    """
    return run(client, sql)
```

Example 5: 12-month moving average

|    | time       | value   | moving_avg_12mo |
|----|------------|---------|-----------------|
| 0  | 2000-01-01 | 100.000 | 100.000000      |
| 1  | 2000-02-01 | 100.571 | 100.285500      |
| 2  | 2000-03-01 | 101.466 | 100.679000      |
| 3  | 2000-04-01 | 102.540 | 101.144250      |
| 4  | 2000-05-01 | 103.701 | 101.655600      |
| 5  | 2000-06-01 | 104.856 | 102.189000      |
| 6  | 2000-07-01 | 105.723 | 102.693857      |
| 7  | 2000-08-01 | 106.523 | 103.172500      |
| 8  | 2000-09-01 | 107.137 | 103.613000      |
| 9  | 2000-10-01 | 107.729 | 104.024600      |
| 10 | 2000-11-01 | 108.292 | 104.412545      |
| 11 | 2000-12-01 | 108.793 | 104.777583      |
| 12 | 2001-01-01 | 109.216 | 105.545583      |
| 13 | 2001-02-01 | 109.643 | 106.301583      |
| 14 | 2001-03-01 | 110.395 | 107.045667      |
| 15 | 2001-04-01 | 111.248 | 107.771333      |
| 16 | 2001-05-01 | 112.203 | 108.479833      |
| 17 | 2001-06-01 | 113.273 | 109.181250      |
| 18 | 2001-07-01 | 114.229 | 109.890083      |
| 19 | 2001-08-01 | 114.990 | 110.595667      |

This query uses a SQL window function to compute a rolling 12-month average over the Case-Shiller index, smoothing short-term noise to reveal the long-term trend. Window functions are fully supported in InfluxDB 3's SQL engine, making it capable of sophisticated analytical queries beyond basic aggregation.

#### Example 6: Inspect Schema

```
def example_inspect_schema(client): 1 usage
    sql = """
        SELECT column_name, data_type
        FROM information_schema.columns
        WHERE table_name = 'housing'
    """
    return run(client, sql)
```

Example 6: schema inspection

|   | column_name | data_type               |
|---|-------------|-------------------------|
| 0 | series      | Dictionary(Int32, Utf8) |
| 1 | time        | Timestamp(ns)           |
| 2 | value       | Float64                 |

This query reads from InfluxDB's `information_schema` to inspect the column names and data types of the housing table. It reveals how InfluxDB stores data internally using tags as dictionary-encoded strings for memory efficiency, timestamps at nanosecond precision, and numeric fields as `Float64`.

**Analysis and Visualization:** With data in InfluxDB, we query all three series and produce visualizations using matplotlib. The `query_all_joined` function pulls each series into a pandas DataFrame for analysis:

```
def query_all_joined(client): 2 usages
    frames = {}
    for tag in ("case_shiller", "mortgage_30yr", "unemployment"):
        df = run(client, sql: f"""
            SELECT time, value
            FROM housing
            WHERE series = '{tag}'
            ORDER BY time
        """)
        frames[tag] = df.set_index("time")["value"]
    wide = pd.DataFrame(frames).sort_index()
    return wide
```

Figure 1 indexes all three series to 100 in January 2000, enabling comparison of growth rates across datasets. Home prices have more than tripled since 2000. Unemployment shows two major crises, including the 2008 financial crisis and the COVID spike in 2020.

Figure 2 plots the Case-Shiller index alongside a 12-month moving average, smoothing short-term noise. The trends include the 2006 pre-crash peak, the 2012 trough, and the explosive post-COVID price surge, which are all clearly visible.

## Strengths and Weaknesses

### Strengths

The biggest strength of InfluxDB is its structure: columnar storage built specifically for time-series data. While relational databases can still store this kind of data, it would be best for situations like the ones mentioned in the Use Cases section to use a data system that is equipped to query and analyze at the fastest speed with time-ordered data. (PeerSpot, 2026) The convenient part of InfluxDB's data format is that it is columnar rather than row-based like relational databases, or document-based in JSON format like MongoDB, which makes it really

fast in searching for the correct value or range of values because the data has already been ordered and corresponds to its timestamp.

In terms of licensing, InfluxDB offers multiple tiers: InfluxDB 3 Core (which is the one we used in the tutorial), InfluxDB Cloud, and InfluxDB 3 Enterprise / Cloud Dedicated. (InfluxData, 2025a) The great part is that it is free, open-source, and easy to download and set up for both individuals and enterprises. The Enterprise tier adds clustering, disaster recovery, and dedicated infrastructure for organizations with higher production demands, though it comes at a cost. (InfluxData, 2025b)

Also, unlike most other NoSQL databases, InfluxDB originally had its own proprietary query language called Flux. However, InfluxDB 3 Core has since adopted SQL (the most widely known data query language) to reduce the learning curve, which makes the entry barrier for this database even easier and friendlier for new users to approach. (InfluxData, 2025c)

In terms of scalability, InfluxDB can write up to 1 million writes per second, which makes it suitable for real-world, high-volume applications like IoT sensor networks, financial monitoring, and large-scale infrastructure. (InfluxData, 2016)

So far, InfluxDB has been quite active in the community. It has over 5,000 companies using it in production and nearly 30,000 stars and 3,600 forks on its GitHub repository, which are great signs of increasing and active adoption. (InfluxData, 2025d) The project also maintains a Discord server, a Community Slack, and a library of community-contributed templates for new users to get started quickly.

## Weaknesses

InfluxDB's main weakness is that it is a specialized database rather than a general-purpose one. It is designed for time-series data, so it works best for information that changes over time, such as housing prices, sensor readings, application metrics, or financial market data. Although InfluxDB 3 supports SQL and can handle some relational-style queries, it is not designed to replace a traditional relational database for applications with many connected entities, such as customers, orders, products, and payments. In many production environments, InfluxDB would likely be used alongside another database, which can add complexity to the overall data infrastructure.

Another weakness is that InfluxDB has changed significantly across versions. Earlier versions relied heavily on InfluxQL; InfluxDB 2 introduced Flux, and InfluxDB 3 now emphasizes SQL and InfluxQL. Because Flux is in maintenance mode and is not supported in InfluxDB 3, teams that invested time in learning Flux may need to rewrite queries or adjust their workflows when moving to newer versions. This creates a potential long-term maintenance issue for teams that want database tools to remain stable across versions. (TDengine, 2025)

The open-source Core version also has limits for production use. InfluxDB 3 Core is useful for learning, development, and real-time monitoring use cases, but InfluxDB Enterprise adds features such as high availability, read replicas, and enhanced security. For larger teams that need stronger reliability, collaboration, or enterprise controls, the free Core version may not be enough. (PeerSpot, 2025)

## Summary

InfluxDB is an excellent database for data where time is a crucial factor and where the goal of data analytics is to quickly search for and compare values across a time range. The low entry barrier of just knowing SQL makes it accessible for a wide range of users, from students

learning data query languages to apply in real-world jobs, to employees at companies dealing with huge volumes of time-stamped data daily.

Our mini project is a good example of this. Three independent time-series datasets came together to display clear trends and correlations between housing prices and key economic indicators in the United States, with straightforward queries and data visualizations that would have required significantly more effort in a general-purpose database.

Of course, there are limitations. As a relatively young NoSQL database, InfluxDB is still actively evolving, and that comes at a cost: long-term users who invested time learning Flux were left behind when version 3 switched entirely to SQL with no migration path. Companies also still need to run InfluxDB alongside other databases to cover general-purpose data needs, which adds infrastructure complexity. But beyond those limitations, with the rising demand for data at scale across large enterprises, having experience with InfluxDB is a valuable skill in key industries like finance, manufacturing, IoT, and DevOps. For any development team or company that works with time-stamped data at scale, whether in finance, manufacturing, or economics like our project, InfluxDB is absolutely worth considering as a primary time-series database.

## References:

- Capital One. (n.d.). *About Capital One*. Capital One Financial Corporation.  
<https://www.capitalone.com/about/>
- InfluxData. (n.d.). *Capital One customer story*. InfluxData.  
<https://www.influxdata.com/customer/capital-one/>
- InfluxData. (n.d.). *Customer case study: Capital One* [PDF]. InfluxData.  
[https://get.influxdata.com/rs/972-GDU-533/images/Customer\\_Case\\_Study\\_Capital\\_One.pdf](https://get.influxdata.com/rs/972-GDU-533/images/Customer_Case_Study_Capital_One.pdf)
- InfluxData. (n.d.). *Tesla customer story*. InfluxData. <https://www.influxdata.com/customer/tesla/>
- InfluxData. (2016). *Getting to 1M values per second on an InfluxDB cluster*. InfluxData Blog.  
<https://www.influxdata.com/blog/getting-to-1m-values-per-second-on-an-influxdb-cluster/>
- InfluxData. (2025). *InfluxDB GitHub repository*. GitHub. <https://github.com/influxdata/influxdb>
- InfluxData. (2025a). *InfluxDB 3 Core*. InfluxData.  
<https://www.influxdata.com/products/influxdb/>
- InfluxData. (2025b). *InfluxDB 3 Core and Enterprise are now in beta*. InfluxData Blog.  
<https://www.influxdata.com/blog/influxdb3-open-source-public-beta/>
- InfluxData. (2025c). *InfluxDB 3 Core and Enterprise GA: The next generation time series platform*. InfluxData Blog. <https://www.influxdata.com/blog/influxdb-3-oss-ga/>
- PeerSpot. (2026). *InfluxDB pros and cons*. PeerSpot.  
<https://www.peerspot.com/products/influxdb-pros-and-cons>
- PeerSpot. (2025). *What needs improvement with InfluxDB?* PeerSpot.  
<https://www.peerspot.com/questions/what-needs-improvement-with-influxdb>
- TDengine. (2025, March 31). *InfluxDB leaves OSS users behind*. TDengine.  
<https://tdengine.com/influxdb-leaves-oss-users-behind/>